

南開大學
Nankai University

计算机学院

计算机组成原理创新实验

8 位 AI 运算器设计 week2

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 4 月 30 日

目录

1 实验目的 **2**

2 实验原理 **2**
2.1 卷积-激活-池化整体流程 2
2.2 3×3 卷积计算方法 2
2.3 ReLU 激活处理 3
2.4 2×2 平均池化计算 3
2.5 运算位宽分析 4

3 整体设计思路 **4**
3.1 从 Week1 基础模块到全链路系统 4
3.2 5×5 输入矩阵与 3×3 卷积核 5
3.3 系统输入输出尺寸分析 5

4 链式运算器模块设计 **6**
4.1 3×3 卷积模块设计 6
4.2 平均池化模块设计 7
4.3 5×5 全链路顶层模块设计 8

5 系统仿真与结果分析 **9**
5.1 仿真输入数据 9
5.2 理论计算结果 10
5.3 Vivado 系统仿真波形 11
5.4 输入输出结果对比 11
5.5 仿真结果分析 12

6 实验总结 **12**

A Verilog 源代码 **13**
A.1 20 位加法器模块代码 13
A.2 3×3 卷积模块代码 13
A.3 2×2 平均池化模块代码 15
A.4 5×5 卷积-激活-池化顶层模块代码 15
A.5 系统仿真测试文件代码 20

1 实验目的

本周实验在第一周基础模块设计的基础上，进一步完成卷积、激活和池化模块的链式集成。第一周已经分别实现并验证了 8 位无符号乘法器、8 位无符号加法器、ReLU 模块以及除以 4 模块，本周主要任务是将这些基础运算单元组合起来，构成一个完整的 8 位定点卷积-激活-池化链式运算器。

本实验面向边缘端 AI 计算场景，模拟智能宠物喂食器中猫狗面部特征提取的基础计算流程。系统输入为一个 5×5 的 8 位无符号灰度矩阵，经过 3×3 卷积运算后得到 3×3 特征矩阵，再经过 ReLU 激活处理，最后通过 2×2 平均池化得到 2×2 输出矩阵。

通过本周实验，需要完成以下目标：

1. 理解 5×5 输入矩阵经过 3×3 卷积后输出尺寸的变化过程；
2. 掌握 3×3 卷积单元中乘法与累加运算的硬件实现方式；
3. 将卷积、ReLU 激活和平均池化模块连接成完整计算链路；
4. 使用 Vivado 对全链路系统进行行为仿真；
5. 对比理论计算结果与仿真输出结果，验证系统功能正确性。

2 实验原理

2.1 卷积-激活-池化整体流程

卷积神经网络中的基础计算流程通常包括卷积层、激活层和池化层。卷积层用于从输入图像中提取局部特征，激活层用于增强网络的非线性表达能力，池化层用于压缩特征图尺寸并降低后续计算量。

本实验采用简化的 CNN 前向计算流程，整体数据流为：

$$5\times5 \text{ 输入矩阵} \rightarrow 3\times3 \text{ 卷积结果} \rightarrow 3\times3 \text{ 激活结果} \rightarrow 2\times2 \text{ 池化输出}$$

其中，输入矩阵为 5×5 的 8 位无符号灰度数据，卷积核大小为 3×3，卷积步长为 1，不进行边界填充。因此，卷积层输出尺寸为 3×3。随后对卷积结果进行 ReLU 激活处理，由于本实验采用无符号数据，激活层在当前条件下等效为直通输出。最后，对 3×3 激活结果进行 2×2 平均池化，池化步长为 1，得到最终的 2×2 输出矩阵。

2.2 3×3 卷积计算方法

3×3 卷积的基本思想是使用一个 3×3 卷积核在输入矩阵上滑动，每次覆盖输入矩阵中的一个 3×3 局部区域。对应位置的输入像素值与卷积核权重相乘，然后将 9 个乘积结果相加，得到一个卷积输出值。

设输入局部矩阵为：

$$\begin{bmatrix} p_{00} & p_{01} & p_{02} \\ p_{10} & p_{11} & p_{12} \\ p_{20} & p_{21} & p_{22} \end{bmatrix}$$

卷积核为：

$$\begin{bmatrix} k_{00} & k_{01} & k_{02} \\ k_{10} & k_{11} & k_{12} \\ k_{20} & k_{21} & k_{22} \end{bmatrix}$$

则一个卷积输出值可以表示为：

$$\begin{aligned} y = & p_{00}k_{00} + p_{01}k_{01} + p_{02}k_{02} \\ & + p_{10}k_{10} + p_{11}k_{11} + p_{12}k_{12} \\ & + p_{20}k_{20} + p_{21}k_{21} + p_{22}k_{22} \end{aligned}$$

本实验输入矩阵大小为 5×5，卷积核大小为 3×3，步长为 1，因此卷积核在水平方向和竖直方向各可以移动 3 个位置，最终得到 3×3 个卷积输出结果。

2.3 ReLU 激活处理

ReLU 激活函数的表达式为：

$$ReLU(x) = max(0, x)$$

其含义是：当输入小于 0 时，输出 0；当输入大于或等于 0 时，输出保持不变。ReLU 可以过滤掉负值特征，只保留正向有效特征。

在一般 CNN 计算中，卷积核权重可能为负数，因此卷积结果也可能出现负数，此时 ReLU 需要进行符号判断。但是本实验中输入像素和卷积核权重均采用 8 位无符号整数表示，卷积乘法和累加结果不会出现负数。因此，在当前实验条件下，ReLU 激活可以简化为直通输出：

$$y = x$$

也就是说，本实验中的 ReLU 模块不需要额外的比较器和选择器，可以直接将卷积结果传递到后续池化模块。这种设计能够减少硬件资源占用，也符合边缘端 AI 运算器低资源、低功耗的设计目标。

2.4 2×2 平均池化计算

池化层用于对特征图进行降采样，在保留主要特征信息的同时减少数据规模。本实验采用 2×2 平均池化，即每次取激活结果中的一个 2×2 区域，将其中 4 个数相加后除以 4，得到一个池化输出值。

设 2×2 池化窗口为：

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

则平均池化输出为：

$$y = \frac{a + b + c + d}{4}$$

在硬件实现中，除以 4 可以等价为二进制右移 2 位：

$$y = (a + b + c + d) >> 2$$

相比直接使用除法器，右移操作结构更简单，资源占用更少。因此，本实验复用第一周设计的除以 4 模块实现平均池化运算。

由于卷积激活后得到的特征矩阵大小为 3×3，采用 2×2 池化窗口且步长为 1 时，池化窗口在水平方向和竖直方向各可以移动 2 个位置，因此最终输出为 2×2 矩阵。

2.5 运算位宽分析

本实验使用 8 位无符号整数作为输入数据和卷积核权重，单个数据的取值范围为：

$$0 \sim 255$$

在卷积过程中，首先需要进行 8 位无符号数乘法。两个 8 位无符号数相乘的最大结果为：

$$255 \times 255 = 65025$$

因此，单个乘法器输出至少需要 16 位才能完整表示结果。

3×3 卷积需要对 9 个乘积结果进行累加。在最极端情况下，9 个乘积结果的最大累加值为：

$$9 \times 65025 = 585225$$

由于：

$$2^{19} = 524288 < 585225 < 2^{20} = 1048576$$

因此，3×3 卷积累加结果至少需要 20 位表示。为避免卷积累加过程中发生溢出，本实验在卷积模块中将乘法结果扩展为 20 位后再进行累加，卷积模块输出设置为 20 位。

在平均池化过程中，输入为 8 位特征值。4 个 8 位数据相加的最大值为：

$$255 + 255 + 255 + 255 = 1020$$

因此池化求和结果至少需要 10 位表示。之后通过右移 2 位完成除以 4 操作，输出仍然保持为 8 位。

综上，本实验主要位宽设置如下：

运算内容	最大可能结果	位宽设置
8 位乘 8 位	65025	16 位
3×3 卷积累加	585225	20 位
2×2 池化求和	1020	10 位
池化输出	255	8 位

表 1: Week2 链式运算器位宽分析

3 整体设计思路

3.1 从 Week1 基础模块到全链路系统

第一周实验主要完成了卷积-激活-池化流程中所需的基础运算模块，包括 8 位无符号乘法器、8 位无符号加法器、ReLU 直通模块以及除以 4 模块。这些模块分别对应 CNN 计算流程中的乘法、累加、

激活和平均池化运算。

第二周实验不再单独验证某一个基础模块，而是将这些基础模块按照 CNN 计算流程连接起来，形成完整的链式运算器。整体计算顺序为：

$$5\times5 \text{ 输入矩阵} \rightarrow 3\times3 \text{ 卷积结果} \rightarrow \text{ReLU 直通} \rightarrow 2\times2 \text{ 平均池化} \rightarrow 2\times2 \text{ 输出矩阵}$$

在具体实现时，首先利用 8 位乘法器完成像素值与卷积核权重的乘法运算，再通过加法器链路将 9 个乘积结果累加，得到单个卷积输出值。由于本实验采用无符号数据，卷积结果不会出现负数，因此 ReLU 激活部分可以继续采用直通结构。最后，将相邻的 4 个激活结果送入平均池化模块，通过求和和右移 2 位得到池化输出。

因此，Week2 的重点不是重新设计基础运算单元，而是完成模块之间的连接和数据流组织，使 Week1 中已经验证正确的基础模块能够共同完成完整的 CNN 前向计算过程。

3.2 5×5 输入矩阵与 3×3 卷积核

为了便于验证系统输出是否正确，本实验在仿真中选取一个简单递增的 5×5 灰度矩阵作为输入。矩阵中每个元素均为 8 位无符号整数，且数值较小，可以避免卷积累加结果超过 8 位输出范围，便于后续观察和分析。

输入矩阵设置为：

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}$$

卷积核采用题目中给出的特征提取卷积核：

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

该卷积核中间位置权重为 2，上、下、左、右位置权重为 1，四个角位置权重为 0。它可以理解为更关注中心像素及其上下左右邻域信息，而忽略窗口四个角上的像素。这样既能够体现卷积核加权求和的特点，又能使手工计算过程比较清晰。

在 Verilog 仿真文件中，输入矩阵元素分别用 `p00` 到 `p44` 表示，卷积核元素分别用 `k00` 到 `k22` 表示。例如，`p00` 表示输入矩阵第 0 行第 0 列元素，`k11` 表示卷积核中心元素。

3.3 系统输入输出尺寸分析

本实验输入为 5×5 矩阵，卷积核大小为 3×3，卷积步长为 1，且不进行边界填充。因此卷积输出尺寸可以表示为：

$$N_{out} = N_{in} - K + 1$$

其中，N_{in} 表示输入尺寸，K 表示卷积核尺寸。代入本实验参数可得：

$$N_{out} = 5 - 3 + 1 = 3$$

因此，5×5 输入矩阵经过 3×3 卷积后，输出为 3×3 卷积结果矩阵。

卷积输出矩阵可以表示为：

$$C = \begin{bmatrix} conv00 & conv01 & conv02 \\ conv10 & conv11 & conv12 \\ conv20 & conv21 & conv22 \end{bmatrix}$$

随后，ReLU 激活层对每个卷积输出元素进行逐点处理。由于本实验中卷积结果均为非负数，因此 ReLU 处理后矩阵尺寸不变，仍为 3×3。

最后，对 3×3 激活结果进行 2×2 平均池化，池化窗口大小为 2×2，步长为 1，不进行填充。池化输出尺寸为：

$$N_{pool} = N_{act} - P + 1$$

其中，N_{act} 表示激活结果矩阵尺寸，P 表示池化窗口尺寸。本实验中 $N_{act} = 3$，$P = 2$，因此：

$$N_{pool} = 3 - 2 + 1 = 2$$

因此，最终输出为 2×2 矩阵：

$$Y = \begin{bmatrix} y00 & y01 \\ y10 & y11 \end{bmatrix}$$

综上，系统输入输出尺寸变化过程为：

$$5 \times 5 \rightarrow 3 \times 3 \rightarrow 3 \times 3 \rightarrow 2 \times 2$$

其中，第一个 3×3 为卷积输出，第二个 3×3 为 ReLU 激活输出，最终 2×2 为平均池化输出。

4 链式运算器模块设计

4.1 3×3 卷积模块设计

3×3 卷积模块是本周实验中最核心的计算单元，其作用是完成一个 3×3 局部输入窗口与一个 3×3 卷积核之间的乘加运算。该模块输入包括 9 个像素数据和 9 个卷积核权重，输出为一个卷积结果。

模块输入可以表示为：

$$\begin{bmatrix} p00 & p01 & p02 \\ p10 & p11 & p12 \\ p20 & p21 & p22 \end{bmatrix}$$

卷积核可以表示为：

$$\begin{bmatrix} k00 & k01 & k02 \\ k10 & k11 & k12 \\ k20 & k21 & k22 \end{bmatrix}$$

卷积模块的输出为：

$$\begin{aligned} y = & p00 \times k00 + p01 \times k01 + p02 \times k02 \\ & + p10 \times k10 + p11 \times k11 + p12 \times k12 \\ & + p20 \times k20 + p21 \times k21 + p22 \times k22 \end{aligned}$$

在硬件实现中，该模块内部主要由两部分组成：乘法部分和累加部分。乘法部分复用第一周设计的 8 位无符号乘法器 `mul8u`，共需要 9 个乘法器分别计算 9 组像素与权重的乘积。由于 8 位无符号数相乘的结果最大为 65025，因此每个乘法器输出为 16 位。

累加部分用于将 9 个 16 位乘积结果相加。考虑到 9 个乘积累加后的最大结果需要 20 位表示，因此在累加前先将 16 位乘积扩展为 20 位，再使用 20 位加法器 `add20u` 完成逐级累加。最终 3×3 卷积模块输出为 20 位结果，可以避免卷积累加过程中发生溢出。

本模块接口设计如下：

信号名称	位宽	说明
`p00` ∼ `p22`	8 位	3×3 输入像素窗口
`k00` ∼ `k22`	8 位	3×3 卷积核权重
`y`	20 位	单个卷积输出结果

表 2: 3×3 卷积模块接口说明

4.2 平均池化模块设计

平均池化模块用于实现 2×2 平均池化操作。该模块输入为 4 个 8 位特征值，输出为 1 个 8 位池化结果。

设输入的 2×2 池化窗口为：

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

则平均池化输出为：

$$y = \frac{a + b + c + d}{4}$$

在 Verilog 实现中，平均池化模块首先复用第一周设计的 8 位加法器 `add8u`，分别计算：

$$sum0 = a + b$$

$$sum1 = c + d$$

由于两个 8 位数相加最大为 510，因此 `sum0` 和 `sum1` 均设置为 9 位。随后将两个 9 位结果扩展后相加，得到 10 位求和结果：

$$sum = sum0 + sum1$$

4 个 8 位数相加的最大值为 1020，需要 10 位表示，因此该位宽设置能够避免池化求和溢出。

最后，平均池化模块复用第一周设计的除以 4 模块 `div4_u10`，通过右移 2 位完成除以 4 操作：

$$y = sum >> 2$$

与直接使用除法运算相比，右移操作结构更简单，更适合硬件实现。

本模块接口设计如下：

信号名称	位宽	说明
`a, b, c, d`	8 位	2×2 池化窗口输入
`sum0, sum1`	9 位	两组 8 位数据相加结果
`sum`	10 位	4 个输入数据的总和
`y`	8 位	平均池化输出结果

表 3: 2×2 平均池化模块接口说明

4.3 5×5 全链路顶层模块设计

全链路顶层模块用于将 3×3 卷积模块、ReLU 激活处理和 2×2 平均池化模块连接起来，完成从 5×5 输入矩阵到 2×2 输出矩阵的完整计算过程。

顶层模块输入为 25 个 8 位像素值，分别表示 5×5 灰度矩阵中的各个元素：

$$\begin{bmatrix} p00 & p01 & p02 & p03 & p04 \\ p10 & p11 & p12 & p13 & p14 \\ p20 & p21 & p22 & p23 & p24 \\ p30 & p31 & p32 & p33 & p34 \\ p40 & p41 & p42 & p43 & p44 \end{bmatrix}$$

同时，顶层模块还输入 9 个 8 位卷积核权重：

$$\begin{bmatrix} k00 & k01 & k02 \\ k10 & k11 & k12 \\ k20 & k21 & k22 \end{bmatrix}$$

由于 5×5 输入矩阵经过 3×3 卷积后会得到 3×3 卷积结果，因此顶层模块内部需要例化 9 个 3×3 卷积模块，分别对应 9 个滑动窗口。卷积结果分别记为：

$$\begin{bmatrix} conv00 & conv01 & conv02 \\ conv10 & conv11 & conv12 \\ conv20 & conv21 & conv22 \end{bmatrix}$$

其中，`conv00` 对应输入矩阵左上角的 3×3 区域，`conv01` 对应窗口向右移动一列后的区域，依此类推。这样可以完成卷积核在 5×5 输入矩阵上的完整滑动过程。

在 ReLU 激活部分，由于本实验中输入像素和卷积核权重均为无符号数，卷积结果不会出现负数，因此 ReLU 激活等效为直通输出。也就是说：

$$actij = convij$$

其中，actij 表示激活后的结果。

之后，顶层模块对 3×3 激活结果进行 2×2 平均池化。由于 3×3 矩阵中可以取出 4 个 2×2 窗口，因此顶层模块内部例化 4 个平均池化模块，分别得到最终输出：

$$\begin{bmatrix} y00 & y01 \\ y10 & y11 \end{bmatrix}$$

各个池化输出的计算区域如下：

$$y00 = avg(conv00, conv01, conv10, conv11)$$

$$y01 = avg(conv01, conv02, conv11, conv12)$$

$$y10 = avg(conv10, conv11, conv20, conv21)$$

$$y11 = avg(conv11, conv12, conv21, conv22)$$

本实验中选取的输入矩阵数值较小，卷积输出结果均小于 255，因此在送入平均池化模块时可以直接取卷积结果的低 8 位作为池化输入。若后续输入数据或卷积核权重增大，导致卷积结果超过 8 位范围，则需要在卷积输出和池化输入之间增加截断或饱和处理模块，以保证系统输出范围符合 8 位定点数要求。

顶层模块整体结构可以概括为：

5×5 输入 → 9 个 3×3 卷积窗口 → 3×3 卷积结果 → ReLU 直通 → 4 个 2×2 池化窗口 → 2×2 输出

因此，该顶层模块完成了本周实验要求的卷积-激活-池化全链路集成。

5 系统仿真与结果分析

5.1 仿真输入数据

为了验证卷积-激活-池化全链路系统是否能够正常工作，本实验在测试文件 tb_cnn_chain_5x5.v 中设置了一个标准的 5×5 灰度输入矩阵。输入矩阵中的数据采用 8 位无符号整数表示，数值从 1 到 25 依次递增，便于后续手工计算和结果对比。

仿真输入矩阵为：

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}$$

仿真中使用的 3×3 卷积核为：

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

该卷积核主要关注窗口中心像素以及其上、下、左、右四个相邻像素。由于卷积核四个角位置权重为 0，因此四个角上的输入像素不会影响当前卷积输出。这样既能体现卷积加权求和的过程，又便于检查仿真结果是否正确。

5.2 理论计算结果

首先计算 3×3 卷积结果。以左上角第一个卷积窗口为例，其输入窗口为：

$$\begin{bmatrix} 1 & 2 & 3 \\ 6 & 7 & 8 \\ 11 & 12 & 13 \end{bmatrix}$$

与卷积核相乘累加可得：

$$conv00 = 1 \times 0 + 2 \times 1 + 3 \times 0 + 6 \times 1 + 7 \times 2 + 8 \times 1 + 11 \times 0 + 12 \times 1 + 13 \times 0$$

$$conv00 = 2 + 6 + 14 + 8 + 12 = 42$$

按照相同方法，对 5×5 输入矩阵中所有 3×3 窗口进行卷积计算，可以得到卷积输出矩阵：

$$C = \begin{bmatrix} 42 & 48 & 54 \\ 72 & 78 & 84 \\ 102 & 108 & 114 \end{bmatrix}$$

由于本实验中输入数据和卷积核权重均为无符号数，卷积结果不会出现负数。因此，经过 ReLU 激活后输出保持不变：

$$A = \begin{bmatrix} 42 & 48 & 54 \\ 72 & 78 & 84 \\ 102 & 108 & 114 \end{bmatrix}$$

接下来对 3×3 激活结果进行 2×2 平均池化。4 个池化输出分别为：

$$y00 = \frac{42 + 48 + 72 + 78}{4} = 60$$

$$y01 = \frac{48 + 54 + 78 + 84}{4} = 66$$

$$y10 = \frac{72 + 78 + 102 + 108}{4} = 90$$

$$y11 = \frac{78 + 84 + 108 + 114}{4} = 96$$

因此，理论上的最终输出矩阵为：

$$Y = \begin{bmatrix} 60 & 66 \\ 90 & 96 \end{bmatrix}$$

5.3 Vivado 系统仿真波形

在 Vivado 中运行 `tb_cnn_chain_5x5.v` 测试文件，对全链路顶层模块进行行为仿真。仿真波形中主要观察 9 个卷积中间结果以及 4 个最终池化输出结果。

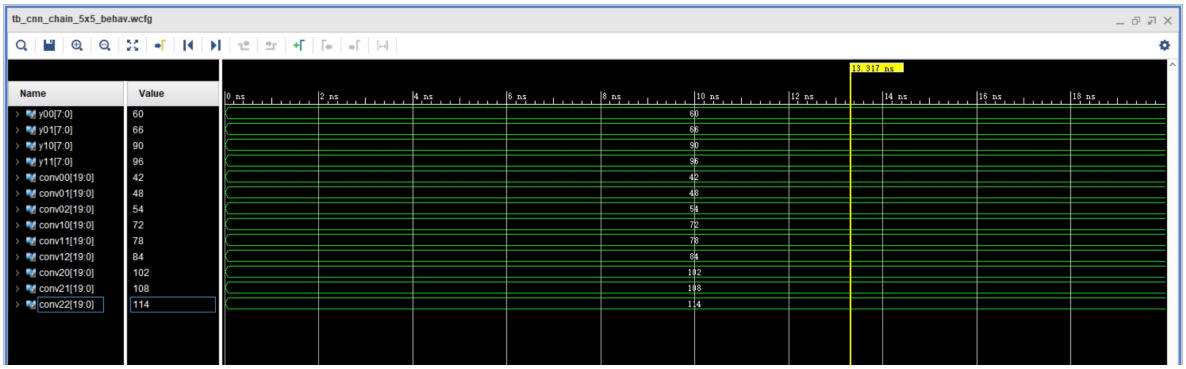


图 5.1: 卷积-激活-池化全链路系统仿真波形

从图5.1可以看出，卷积模块输出结果分别为：

$$42,\ 48,\ 54,\ 72,\ 78,\ 84,\ 102,\ 108,\ 114$$

最终池化输出结果分别为：

$$60,\ 66,\ 90,\ 96$$

这些结果与理论计算结果一致，说明 3×3 卷积、ReLU 激活以及 2×2 平均池化之间的数据连接正确。

5.4 输入输出结果对比

为了更加直观地验证系统功能，下面将理论计算结果与 Vivado 仿真结果进行对比。

信号名称	含义	理论值	仿真值
conv00	第 1 行第 1 列卷积输出	42	42
conv01	第 1 行第 2 列卷积输出	48	48
conv02	第 1 行第 3 列卷积输出	54	54
conv10	第 2 行第 1 列卷积输出	72	72
conv11	第 2 行第 2 列卷积输出	78	78
conv12	第 2 行第 3 列卷积输出	84	84
conv20	第 3 行第 1 列卷积输出	102	102
conv21	第 3 行第 2 列卷积输出	108	108
conv22	第 3 行第 3 列卷积输出	114	114
y00	第 1 行第 1 列池化输出	60	60
y01	第 1 行第 2 列池化输出	66	66
y10	第 2 行第 1 列池化输出	90	90
y11	第 2 行第 2 列池化输出	96	96

表 4: 全链路系统理论结果与仿真结果对比

由表4可知，所有卷积输出和最终池化输出的仿真值均与理论值一致，说明本实验设计的链式运算器能够正确完成预期计算。

5.5 仿真结果分析

本次仿真验证了从 5×5 输入矩阵到 2×2 输出矩阵的完整计算过程。首先，系统通过 9 个 3×3 卷积窗口分别计算得到 3×3 卷积结果。仿真波形中 conv00 到 conv22 的值依次为 42、48、54、72、78、84、102、108、114，与手工计算得到的卷积矩阵完全一致。

其次，由于本实验中卷积核和输入矩阵均采用无符号整数表示，卷积结果不存在负数，因此 ReLU 激活层不会改变数据值。也就是说，激活后的结果仍然等于卷积结果。这一点与前面模块设计中的 ReLU 直通逻辑一致。

最后，系统对 3×3 卷积结果进行 2×2 平均池化，得到最终输出矩阵：

$$\begin{bmatrix} 60 & 66 \\ 90 & 96 \end{bmatrix}$$

从仿真波形可以看出，y00、y01、y10、y11 分别为 60、66、90、96，与理论池化结果一致。

综上，本周完成了 3×3 卷积单元以及卷积-激活-池化全链路系统的集成，并通过标准 5×5 灰度矩阵完成了系统仿真。仿真结果表明，各级模块之间连接正确，系统能够按照预期完成 CNN 基础计算流程。

6 实验总结

本周实验在第一周基础运算模块的基础上，完成了 8 位定点卷积-激活-池化链式运算器的系统集成与功能仿真。相比第一周只对乘法器、加法器、ReLU 模块和除以 4 模块进行单独验证，本周进一步将这些模块连接成完整的数据通路，实现了从 5×5 灰度矩阵输入到 2×2 特征矩阵输出的完整计算过程。

在 3×3 卷积模块设计中，本实验使用 9 个 8 位无符号乘法器完成像素值与卷积核权重的乘法运算，并通过 20 位累加结构完成 9 项乘积求和。由于 3×3 卷积累加结果可能超过 8 位范围，因此在卷积模块中对中间结果进行了位宽扩展，避免了累加过程中的数据溢出。

在全链路系统设计中，5×5 输入矩阵经过 9 个 3×3 滑动窗口计算后，得到 3×3 卷积输出矩阵。由于本实验采用无符号输入和无符号卷积核，卷积结果不会出现负数，因此 ReLU 激活层在当前条件下等效为直通输出。随后，系统对 3×3 激活结果进行 2×2 平均池化，最终得到 2×2 输出矩阵。

通过 Vivado 行为仿真可以看出，卷积输出结果为：

$$\begin{bmatrix} 42 & 48 & 54 \\ 72 & 78 & 84 \\ 102 & 108 & 114 \end{bmatrix}$$

最终池化输出结果为：

$$\begin{bmatrix} 60 & 66 \\ 90 & 96 \end{bmatrix}$$

仿真结果与理论计算结果完全一致，说明本周设计的链式运算器能够正确完成 3×3 卷积、ReLU 激活以及 2×2 平均池化计算。至此，本实验完成了从基础运算模块到完整 CNN 计算链路的集成，为后续进一步优化资源占用、降低计算时延和扩展更复杂的卷积运算结构奠定了基础。

附录 A Verilog 源代码

本周代码在 Week1 基础模块的基础上完成系统集成。其中，`mul8u`、`add8u`、`div4_u10` 等基础模块已在 Week1 附录中给出，本附录仅列出 Week2 新增或扩展的 20 位加法器、卷积模块、池化模块、顶层模块和系统测试文件。

A.1 20 位加法器模块代码

```
module add20u (
    input [19:0] a,
    input [19:0] b,
    output [19:0] y
);
assign y = a + b;
endmodule
```

A.2 3×3 卷积模块代码

```
module conv3x3_u8 (
    input [7:0] p00, p01, p02,
    input [7:0] p10, p11, p12,
    input [7:0] p20, p21, p22,

    input [7:0] k00, k01, k02,
    input [7:0] k10, k11, k12,
    input [7:0] k20, k21, k22,

```

```
10      output [19:0] y
11  );
12
13  wire [15:0] m00_16, m01_16, m02_16;
14  wire [15:0] m10_16, m11_16, m12_16;
15  wire [15:0] m20_16, m21_16, m22_16;
16
17  wire [19:0] m00, m01, m02;
18  wire [19:0] m10, m11, m12;
19  wire [19:0] m20, m21, m22;
20
21  wire [19:0] s0, s1, s2, s3, s4, s5, s6;
22
23  // 9个乘法
24  mul8u u_mul00 (.a(p00), .b(k00), .y(m00_16));
25  mul8u u_mul01 (.a(p01), .b(k01), .y(m01_16));
26  mul8u u_mul02 (.a(p02), .b(k02), .y(m02_16));
27
28  mul8u u_mul10 (.a(p10), .b(k10), .y(m10_16));
29  mul8u u_mul11 (.a(p11), .b(k11), .y(m11_16));
30  mul8u u_mul12 (.a(p12), .b(k12), .y(m12_16));
31
32  mul8u u_mul20 (.a(p20), .b(k20), .y(m20_16));
33  mul8u u_mul21 (.a(p21), .b(k21), .y(m21_16));
34  mul8u u_mul22 (.a(p22), .b(k22), .y(m22_16));
35
36  // 16位扩展为20位
37  assign m00 = {4'b0000, m00_16};
38  assign m01 = {4'b0000, m01_16};
39  assign m02 = {4'b0000, m02_16};
40
41  assign m10 = {4'b0000, m10_16};
42  assign m11 = {4'b0000, m11_16};
43  assign m12 = {4'b0000, m12_16};
44
45  assign m20 = {4'b0000, m20_16};
46  assign m21 = {4'b0000, m21_16};
47  assign m22 = {4'b0000, m22_16};
48
49  // 9项累加
50  add20u u_add0 (.a(m00), .b(m01), .y(s0));
51  add20u u_add1 (.a(s0),  .b(m02), .y(s1));
52  add20u u_add2 (.a(s1),  .b(m10), .y(s2));
53  add20u u_add3 (.a(s2),  .b(m11), .y(s3));
54  add20u u_add4 (.a(s3),  .b(m12), .y(s4));
55  add20u u_add5 (.a(s4),  .b(m20), .y(s5));
56  add20u u_add6 (.a(s5),  .b(m21), .y(s6));
57  add20u u_add7 (.a(s6),  .b(m22), .y(y));
58
```

```
endmodule
```

A.3 2×2 平均池化模块代码

```
module avg_pool2x2_u8 (
    input  [7:0] a,
    input  [7:0] b,
    input  [7:0] c,
    input  [7:0] d,
    output [7:0] y
);

wire [8:0] sum0;
wire [8:0] sum1;
wire [9:0] sum;

add8u u_add0 (.a(a), .b(b), .y(sum0));
add8u u_add1 (.a(c), .b(d), .y(sum1));

assign sum = {1'b0, sum0} + {1'b0, sum1};

div4_u10 u_div4 (
    .x(sum),
    .y(y)
);

endmodule
```

A.4 5×5 卷积-激活-池化顶层模块代码

```
module mul8u (
    input  [7:0] a,
    input  [7:0] b,
    output [15:0] y
);
assign y = a * b;
endmodule


module add8u (
    input  [7:0] a,
    input  [7:0] b,
    output [8:0] y
);
assign y = a + b;
endmodule
```

```
17
18
19 module div4_u10 (
20     input  [9:0] x,
21     output [7:0] y
22 );
23 assign y = x >> 2;
24 endmodule
25
26
27 module add20u (
28     input  [19:0] a,
29     input  [19:0] b,
30     output [19:0] y
31 );
32 assign y = a + b;
33 endmodule
34
35
36 module conv3x3_u8 (
37     input [7:0] p00, p01, p02,
38     input [7:0] p10, p11, p12,
39     input [7:0] p20, p21, p22,
40
41     input [7:0] k00, k01, k02,
42     input [7:0] k10, k11, k12,
43     input [7:0] k20, k21, k22,
44
45     output [19:0] y
46 );
47
48 wire [15:0] m00_16, m01_16, m02_16;
49 wire [15:0] m10_16, m11_16, m12_16;
50 wire [15:0] m20_16, m21_16, m22_16;
51
52 wire [19:0] m00, m01, m02;
53 wire [19:0] m10, m11, m12;
54 wire [19:0] m20, m21, m22;
55
56 wire [19:0] s0, s1, s2, s3, s4, s5, s6;
57
58 mul8u u_mul00 (.a(p00), .b(k00), .y(m00_16));
59 mul8u u_mul01 (.a(p01), .b(k01), .y(m01_16));
60 mul8u u_mul02 (.a(p02), .b(k02), .y(m02_16));
61
62 mul8u u_mul10 (.a(p10), .b(k10), .y(m10_16));
63 mul8u u_mul11 (.a(p11), .b(k11), .y(m11_16));
64 mul8u u_mul12 (.a(p12), .b(k12), .y(m12_16));
65
```

```
66  mul8u u_mul20 (.a(p20), .b(k20), .y(m20_16));
67  mul8u u_mul21 (.a(p21), .b(k21), .y(m21_16));
68  mul8u u_mul22 (.a(p22), .b(k22), .y(m22_16));
69
70  assign m00 = {4'b0000, m00_16};
71  assign m01 = {4'b0000, m01_16};
72  assign m02 = {4'b0000, m02_16};
73
74  assign m10 = {4'b0000, m10_16};
75  assign m11 = {4'b0000, m11_16};
76  assign m12 = {4'b0000, m12_16};
77
78  assign m20 = {4'b0000, m20_16};
79  assign m21 = {4'b0000, m21_16};
80  assign m22 = {4'b0000, m22_16};
81
82  add20u u_add0 (.a(m00), .b(m01), .y(s0));
83  add20u u_add1 (.a(s0),  .b(m02), .y(s1));
84  add20u u_add2 (.a(s1),  .b(m10), .y(s2));
85  add20u u_add3 (.a(s2),  .b(m11), .y(s3));
86  add20u u_add4 (.a(s3),  .b(m12), .y(s4));
87  add20u u_add5 (.a(s4),  .b(m20), .y(s5));
88  add20u u_add6 (.a(s5),  .b(m21), .y(s6));
89  add20u u_add7 (.a(s6),  .b(m22), .y(y));
90
91  endmodule
92
93
94  module avg_pool2x2_u8 (
95      input  [7:0] a,
96      input  [7:0] b,
97      input  [7:0] c,
98      input  [7:0] d,
99      output [7:0] y
100 );
101
102 wire [8:0] sum0;
103 wire [8:0] sum1;
104 wire [9:0] sum;
105
106 add8u u_add0 (.a(a), .b(b), .y(sum0));
107 add8u u_add1 (.a(c), .b(d), .y(sum1));
108
109 assign sum = {1'b0, sum0} + {1'b0, sum1};
110
111 div4_u10 u_div4 (
112     .x(sum),
113     .y(y)
114 );
```

```
115
116 endmodule
117
118
119 module cnn_chain_5x5 (
120     input [7:0] p00, p01, p02, p03, p04,
121     input [7:0] p10, p11, p12, p13, p14,
122     input [7:0] p20, p21, p22, p23, p24,
123     input [7:0] p30, p31, p32, p33, p34,
124     input [7:0] p40, p41, p42, p43, p44,
125
126     input [7:0] k00, k01, k02,
127     input [7:0] k10, k11, k12,
128     input [7:0] k20, k21, k22,
129
130     output [7:0] y00,
131     output [7:0] y01,
132     output [7:0] y10,
133     output [7:0] y11
134 );
135
136 wire [19:0] conv00, conv01, conv02;
137 wire [19:0] conv10, conv11, conv12;
138 wire [19:0] conv20, conv21, conv22;
139
140 conv3x3_u8 u_conv00 (
141     .p00(p00), .p01(p01), .p02(p02),
142     .p10(p10), .p11(p11), .p12(p12),
143     .p20(p20), .p21(p21), .p22(p22),
144     .k00(k00), .k01(k01), .k02(k02),
145     .k10(k10), .k11(k11), .k12(k12),
146     .k20(k20), .k21(k21), .k22(k22),
147     .y(conv00)
148 );
149
150 conv3x3_u8 u_conv01 (
151     .p00(p01), .p01(p02), .p02(p03),
152     .p10(p11), .p11(p12), .p12(p13),
153     .p20(p21), .p21(p22), .p22(p23),
154     .k00(k00), .k01(k01), .k02(k02),
155     .k10(k10), .k11(k11), .k12(k12),
156     .k20(k20), .k21(k21), .k22(k22),
157     .y(conv01)
158 );
159
160 conv3x3_u8 u_conv02 (
161     .p00(p02), .p01(p03), .p02(p04),
162     .p10(p12), .p11(p13), .p12(p14),
163     .p20(p22), .p21(p23), .p22(p24),
```

```
164     .k00(k00), .k01(k01), .k02(k02),
165     .k10(k10), .k11(k11), .k12(k12),
166     .k20(k20), .k21(k21), .k22(k22),
167     .y(conv02)
168 );
169 
170 conv3x3_u8 u_conv10 (
171     .p00(p10), .p01(p11), .p02(p12),
172     .p10(p20), .p11(p21), .p12(p22),
173     .p20(p30), .p21(p31), .p22(p32),
174     .k00(k00), .k01(k01), .k02(k02),
175     .k10(k10), .k11(k11), .k12(k12),
176     .k20(k20), .k21(k21), .k22(k22),
177     .y(conv10)
178 );
179 
180 conv3x3_u8 u_conv11 (
181     .p00(p11), .p01(p12), .p02(p13),
182     .p10(p21), .p11(p22), .p12(p23),
183     .p20(p31), .p21(p32), .p22(p33),
184     .k00(k00), .k01(k01), .k02(k02),
185     .k10(k10), .k11(k11), .k12(k12),
186     .k20(k20), .k21(k21), .k22(k22),
187     .y(conv11)
188 );
189 
190 conv3x3_u8 u_conv12 (
191     .p00(p12), .p01(p13), .p02(p14),
192     .p10(p22), .p11(p23), .p12(p24),
193     .p20(p32), .p21(p33), .p22(p34),
194     .k00(k00), .k01(k01), .k02(k02),
195     .k10(k10), .k11(k11), .k12(k12),
196     .k20(k20), .k21(k21), .k22(k22),
197     .y(conv12)
198 );
199 
200 conv3x3_u8 u_conv20 (
201     .p00(p20), .p01(p21), .p02(p22),
202     .p10(p30), .p11(p31), .p12(p32),
203     .p20(p40), .p21(p41), .p22(p42),
204     .k00(k00), .k01(k01), .k02(k02),
205     .k10(k10), .k11(k11), .k12(k12),
206     .k20(k20), .k21(k21), .k22(k22),
207     .y(conv20)
208 );
209 
210 conv3x3_u8 u_conv21 (
211     .p00(p21), .p01(p22), .p02(p23),
212     .p10(p31), .p11(p32), .p12(p33),
```

```
    .p20(p41), .p21(p42), .p22(p43),
    .k00(k00), .k01(k01), .k02(k02),
    .k10(k10), .k11(k11), .k12(k12),
    .k20(k20), .k21(k21), .k22(k22),
    .y(conv21)
);

conv3x3_u8 u_conv22 (
    .p00(p22), .p01(p23), .p02(p24),
    .p10(p32), .p11(p33), .p12(p34),
    .p20(p42), .p21(p43), .p22(p44),
    .k00(k00), .k01(k01), .k02(k02),
    .k10(k10), .k11(k11), .k12(k12),
    .k20(k20), .k21(k21), .k22(k22),
    .y(conv22)
);

// 本实验使用无符号卷积结果，因此ReLU等效为直通。
// 当前测试数据的卷积结果均小于255，因此可直接取低8位进入池化。
avg_pool2x2_u8 u_pool00 (
    .a(conv00[7:0]), .b(conv01[7:0]),
    .c(conv10[7:0]), .d(conv11[7:0]),
    .y(y00)
);

avg_pool2x2_u8 u_pool01 (
    .a(conv01[7:0]), .b(conv02[7:0]),
    .c(conv11[7:0]), .d(conv12[7:0]),
    .y(y01)
);

avg_pool2x2_u8 u_pool10 (
    .a(conv10[7:0]), .b(conv11[7:0]),
    .c(conv20[7:0]), .d(conv21[7:0]),
    .y(y10)
);

avg_pool2x2_u8 u_pool11 (
    .a(conv11[7:0]), .b(conv12[7:0]),
    .c(conv21[7:0]), .d(conv22[7:0]),
    .y(y11)
);

endmodule
```

A.5 系统仿真测试文件代码

```
`timescale 1ns/1ps
```

```
2
3  module tb_cnn_chain_5x5;
4
5  reg [7:0] p00, p01, p02, p03, p04;
6  reg [7:0] p10, p11, p12, p13, p14;
7  reg [7:0] p20, p21, p22, p23, p24;
8  reg [7:0] p30, p31, p32, p33, p34;
9  reg [7:0] p40, p41, p42, p43, p44;
10
11 reg [7:0] k00, k01, k02;
12 reg [7:0] k10, k11, k12;
13 reg [7:0] k20, k21, k22;
14
15 wire [7:0] y00, y01, y10, y11;
16
17 cnn_chain_5x5 uut (
18     .p00(p00), .p01(p01), .p02(p02), .p03(p03), .p04(p04),
19     .p10(p10), .p11(p11), .p12(p12), .p13(p13), .p14(p14),
20     .p20(p20), .p21(p21), .p22(p22), .p23(p23), .p24(p24),
21     .p30(p30), .p31(p31), .p32(p32), .p33(p33), .p34(p34),
22     .p40(p40), .p41(p41), .p42(p42), .p43(p43), .p44(p44),
23
24     .k00(k00), .k01(k01), .k02(k02),
25     .k10(k10), .k11(k11), .k12(k12),
26     .k20(k20), .k21(k21), .k22(k22),
27
28     .y00(y00), .y01(y01),
29     .y10(y10), .y11(y11)
30 );
31
32 initial begin
33     // 5x5 input matrix
34     p00 = 8'd1;   p01 = 8'd2;   p02 = 8'd3;   p03 = 8'd4;   p04 = 8'd5;
35     p10 = 8'd6;   p11 = 8'd7;   p12 = 8'd8;   p13 = 8'd9;   p14 = 8'd10;
36     p20 = 8'd11;  p21 = 8'd12;  p22 = 8'd13;  p23 = 8'd14;  p24 = 8'd15;
37     p30 = 8'd16;  p31 = 8'd17;  p32 = 8'd18;  p33 = 8'd19;  p34 = 8'd20;
38     p40 = 8'd21;  p41 = 8'd22;  p42 = 8'd23;  p43 = 8'd24;  p44 = 8'd25;
39
40     // 3x3 convolution kernel
41     k00 = 8'd0;   k01 = 8'd1;   k02 = 8'd0;
42     k10 = 8'd1;   k11 = 8'd2;   k12 = 8'd1;
43     k20 = 8'd0;   k21 = 8'd1;   k22 = 8'd0;
44
45     #10;
46
47     $display("Convolution output:");
48     $display("%0d %0d %0d", uut.conv00, uut.conv01, uut.conv02);
49     $display("%0d %0d %0d", uut.conv10, uut.conv11, uut.conv12);
50     $display("%0d %0d %0d", uut.conv20, uut.conv21, uut.conv22);
```

```
51
52      $display("Pooling output:");
53      $display("%0d %0d", y00, y01);
54      $display("%0d %0d", y10, y11);
55
56      #10;
57      $finish;
58  end
59
60  endmodule
```